

Rapport de projet

Titre Professionnel Concepteur Développeur d'Applications

Lifequest - Application de gestion financière personnelle

Candidat : Antonin MINGAM

Titre visé : Concepteur Développeur d'Applications Niveau 6

Référentiel : TP-01281 millésime 04 (24/05/2023)

Sommaire

1. [Présentation du candidat et du contexte de formation](#)
2. [Présentation du projet Lifequest](#)
3. 2.1 [Contexte et objectifs](#)
4. 2.2 [Stack technique](#)
5. 2.3 [Architecture générale](#)

BLOC 1 — Développer une application sécurisée

1. [CP1 — Installer et configurer son environnement de travail en fonction du projet](#)
2. 3.1 [Environnement de développement](#)
3. 3.2 [Outils de gestion des versions et de collaboration](#)
4. 3.3 [Conteneurisation](#)
5. [CP2 — Développer des interfaces utilisateur](#)
6. 4.1 [Conception des interfaces avec Phoenix LiveView](#)
7. 4.2 [Charte graphique et responsive design \(Tailwind CSS / daisyUI\)](#)
8. 4.3 [Accessibilité \(RGAA\)](#)
9. 4.4 [Conformité RGPD — Mentions légales](#)
10. 4.5 [Tests des composants d'interface](#)
11. 4.6 [Sécurité des interfaces \(XSS, CSRF\)](#)
12. [CP3 — Développer des composants métier](#)
13. 5.1 [Architecture en contextes \(Finances, Accounts\)](#)
14. 5.2 [Développement défensif et gestion des erreurs](#)
15. 5.3 [Communication en temps réel avec Phoenix PubSub](#)
16. 5.4 [Tests unitaires des composants métier](#)
17. 5.5 [Sécurité des composants métier](#)
18. [CP4 — Contribuer à la gestion d'un projet informatique](#)
19. 6.1 [Méthode de développement itérative](#)

- 20. 6.2 [Planification et suivi des tâches \(Jira\)](#)
- 21. 6.3 [Outils collaboratifs \(GitHub, conventions de commits\)](#)
- 22. 6.4 [Qualité du code \(Credo, mix format, CI\)](#)

BLOC 2 — Concevoir et développer une application sécurisée organisée en couches

- 1. [CP5 — Analyser les besoins et maquetter une application](#)
- 2. 7.1 [Analyse du cahier des charges et identification des besoins](#)
- 3. 7.2 [User stories](#)
- 4. 7.3 [Maquettes et enchaînement des écrans](#)
- 5. 7.4 [Dossier de conception](#)
- 6. [CP6 — Définir l'architecture logicielle d'une application](#)
- 7. 8.1 [Architecture multicouche \(web / contextes / schémas\)](#)
- 8. 8.2 [Choix du framework et de l'ORM \(Phoenix / Ecto\)](#)
- 9. 8.3 [Stratégie de sécurité par couche](#)
- 10. 8.4 [Éco-conception](#)
- 11. [CP7 — Concevoir et mettre en place une base de données relationnelle](#)
- 12. 9.1 [Modèle Conceptuel de Données \(MCD\)](#)
- 13. 9.2 [Modèle Physique de Données \(MPD\)](#)
- 14. 9.3 [Migrations Ecto et scripts de création](#)
- 15. 9.4 [Sécurité et droits d'accès](#)
- 16. 9.5 [Jeu d'essai](#)
- 17. [CP8 — Développer des composants d'accès aux données SQL et NoSQL](#)
 - 10.1 [Requêtes Ecto \(CRUD sécurisé\)](#)
 - 10.2 [Gestion de l'intégrité et des conflits d'accès \(transactions\)](#)
 - 10.3 [Validation et contrôle des entrées côté serveur](#)
 - 10.4 [Tests unitaires et de sécurité des accès aux données](#)
 - 10.5 [Protection contre l'injection SQL](#)

BLOC 3 — Préparer le déploiement d'une application sécurisée

1. [CP9 — Préparer et exécuter les plans de tests d'une application](#)
 - 11.1 [Plan de tests](#)
 - 11.2 [Tests unitaires \(ExUnit\)](#)
 - 11.3 [Tests d'intégration \(LiveView\)](#)
 - 11.4 [Tests de sécurité](#)
 - 11.5 [Résultats et compte-rendu de tests](#)
2. [CP10 — Préparer et documenter le déploiement d'une application](#)
 - 12.1 [Environnements \(développement, test, production\)](#)
 - 12.2 [Procédure de déploiement](#)
 - 12.3 [Scripts de déploiement et migrations](#)
 - 12.4 [Variables d'environnement et configuration](#)
3. [CP11 — Contribuer à la mise en production dans une démarche DevOps](#)
 - 13.1 [Pipeline CI/CD \(GitHub Actions\)](#)
 - 13.2 [Conteneurisation \(Docker\)](#)
 - 13.3 [Outils de qualité de code \(Credo, mix format\)](#)
 - 13.4 [Automatisation des tests](#)
 - 13.5 [Interprétation des rapports CI](#)

Compétences transversales

1. [Communiquer en français et en anglais](#)
 - 14.1 [Documentation technique en anglais](#)
 - 14.2 [Lecture de documentation technique en anglais \(niveau B1\)](#)
2. [Mettre en œuvre une démarche de résolution de problème](#)
3. [Apprendre en continu — Veille technologique](#)
4. [Conclusion et bilan de compétences](#)
5. [Annexes](#)
 - A. Diagramme MCD / MPD

- B. Maquettes des interfaces
- C. Plan de tests complet
- D. Extraits de code commentés
- E. Captures d'écran de l'application

1. Présentation du candidat et du contexte de formation

Avant d'entrer en formation, mon parcours étudiant et professionnel ont été un long processus exploratoire. Après une scolarité exemplaire, j'ai traversé plusieurs filières "sérieuse" avant de succomber au choix du coeur. J'ai finalement jeté mon dévolu sur une carrière mêlant technique et artistique, en tant qu'ingénieur du son. J'ai exercé cette activité pendant 5 ans en autoentreprise, acquérant au passage des compétences techniques et transversales, d'autonomie et de gestion d'entreprise. Mon choix de changer de direction arrive à la rencontre de plusieurs évènements simultanés dans ma vie d'entrepreneur :

- la sensation d'avoir atteint un plafond de verre
- la frustration de ne pas avoir de diplôme qualifiant
- l'arrivée imminente de mes 30 ans
- la rencontre avec le monde du développement En effet, malgré une rapide découverte au cours de mon adolescence, je n'avais jamais vraiment compris ce monde si vaste et riche. Mais lors de mon parcours, j'ai été amené à rédiger des newsletters en HTML/CSS d'abord, puis à vouloir refondre mon site web professionnel qui avait été créé grâce à un outil no-code à l'origine. J'ai fini par consacrer une grande partie de mon temps libre à l'apprentissage des bases du développement, grâce aux cours en ligne d'OpenClassrooms puis de l'université d'Harvard. Cette montée en compétences progressive m'a convaincu de mon attrait pour le développement, et de ma motivation à concrétiser ces nouvelles compétences grâce à des études traditionnelles. C'est dans la suite logique de ma formation en autodidacte que j'ai décidé d'intégrer la 2iAcademy en Conception et Développement d'Applications et en alternance chez Frixel afin de renforcer mes compétences et d'en acquérir de nouvelles en conception, ainsi que d'obtenir un diplôme reconnu et qualifiant. À l'issue de cette formation, je poursuivrai au sein de l'ESGI Paris en Mastère Big Data & Intelligence Artificielle dans le but de me spécialiser dans un

secteur en forte croissance. À plus long terme, je me réserve le droit de retrouver le chemin de l'entrepreneuriat, fort cette fois d'une double compétence technique et d'un diplôme reconnu.

2. Présentation du projet Lifequest

2.1 Contexte et objectifs

Durant mon activité entrepreneuriale, il était très compliqué de me projeter dans l'avenir à moyen ou long terme. Mon arrivée dans un univers professionnel stable m'a permis de découvrir la gestion d'épargne à long terme, les objectifs financiers et les placements financiers. En explorant le sujet, j'ai découvert beaucoup d'applications de gestion financière et de placements financiers, mais aucune qui permettait de faire les deux à la fois, en plus d'offrir du conseil en gestion de patrimoine. Un ami travaillant dans la finance m'avait fait part de son idée d'application de gestion et de conseil financiers, et une connaissance japonaise m'a fait découvrir des outils de gestion financière répandus au Japon. Ces différents échanges ont fait émerger l'idée de Lifequest, avec une approche innovante articulée autour des objectifs de vie de l'utilisateur, comme un achat de maison ou la volonté d'avoir un enfant. L'utilisateur renseigne sa situation financière actuelle, une estimation de ses revenus et de ses dépenses mensuels. Cela nous permet de prendre une photo de son patrimoine et d'effectuer des projections du futur simplement. Il définit ensuite un objectif représenté par un montant cible et une échéance. À partir de ces données, l'application génère des recommandations personnalisées : réduction des dépenses non essentielles, placements ponctuels ou réguliers, avec un niveau de risque adapté à la complexité de l'objectif et à la situation financière déclarée.

L'application cible des particuliers souhaitant gérer leur budget personnel et planifier des objectifs financiers à moyen ou long terme, sans nécessiter de connaissances préalables en finance.

2.2 Stack technique

- Backend : Elixir 1.18 / OTP 27, Phoenix 1.8, Phoenix LiveView 1.1, Ecto
- Base de données : PostgreSQL 16

- Frontend : Tailwind CSS v4, daisyUI, templates HEx (pas de framework JS séparé)
- Tests : ExUnit, ConnCase, Phoenix.LiveViewTest
- CI : GitHub Actions

Le choix de cette stack technique s'est présentée comme une évidence, car c'est celle que j'ai apprise et sur laquelle j'ai travaillé durant la majorité de mon alternance chez Frixel. Le framework Phoenix est très complet, innovant, et bien conçu. La séparation des responsabilités est très claire (contexts, schemas, controllers, LiveViews), les outils intégrés sont très riches, et LiveView permet de construire des interfaces réactives côté serveur sans avoir à maintenir un framework JavaScript séparé.

Côté données, Phoenix fournit l'ORM Ecto, conçu pour fonctionner par défaut avec PostgreSQL, avec un système de migrations versionné et les changesets qui permettent la validation des données avant toute écriture en base.

`mix phx.gen.auth` nous fournit un système d'authentification poussé en une commande, avec stockage des mots de passes hashés avec bcrypt en base, ainsi que le magic link, un système de vérification de compte et de réinitialisation de mot de passe grâce à un envoi de token par email.

Les tests s'appuient sur ExUnit pour les tests unitaires, ConnCase pour les routes HTTP et Phoenix.LiveViewTest pour les interactions LiveView.

L'intégration continue est assurée par GitHub Actions, dont le pipeline exécute les tâches d'installation et de validation fournies par l'écosystème Phoenix : `mix format --check-formatted` et `mix credo --strict` pour le linting et le formatage, `mix test` pour l'exécution des tests et `mix compile --warnings-as-errors` pour la compilation stricte du projet.

2.3 Architecture générale

Lifequest repose sur une architecture en trois couches strictement séparées, afin de respecter la convention proposée par le framework.

Couche présentation : HEx est un langage de templating permettant de construire nos pages webs en Elixir. Ce sont les LiveViews qui sont chargées

d'interpréter ce code. Lors de leur création, un état est généré côté serveur et une Websocket persistante est ouverte pour communiquer avec le navigateur. Les interactions utilisateur déclenchent des événements traités par le serveur qui renvoie le diff partiel du DOM, permettant de changer l'interface sans rechargement de la page.

Couche métier : Ce sont les contextes Elixir qui portent ces responsabilités. Chaque contexte expose des fonctions publiques qui pourront être appelées par l'ensemble des LiveView. Ces dernières doivent passer par les fonctions des contextes pour interagir avec la base de données. Cela centralise la logique métier, et facilite les tests.

Couche données : Les schémas Ecto définissent la structure des données et les changesets qui valident les entrées avant toute écriture. Ecto génère des requêtes paramétrées afin d'éviter les injections SQL.

Une autre fonctionnalité offerte par Phoenix s'appelle Scope. Il s'agit d'une struct représentant l'utilisateur actuel qui est passée en premier argument à toutes les fonctions de contexte. Cela garantit l'isolation des données entre les utilisateurs, car elle est utilisée comme filtre dans toutes les requêtes Ecto. On peut également s'appuyer sur elle pour la gestion des droits, par exemple entre admin et user, ou entre compte free ou premium.

Pour résumer, à l'appel d'une route : **Voir Annexe 1.1**

- la requête HTTP initiale déclenche le `mount/3` de la LiveView
- les fonctions de contextes nécessaires au rendu initial sont appelées
- les requêtes SQL sont envoyées à PostgreSQL grâce à Ecto
- les données sont placées en `assigns` sur la socket et stockées en mémoire vive
- le template HEx s'affiche, en utilisant ces données

Par la suite, les interactions utilisateurs déclencheront des fonctions `handle_event/3` qui fonctionnent de manière similaire, et qui permettront de modifier le DOM sans rechargement ou d'émettre des messages PubSub pour avertir d'autres LiveViews abonnées de la mise à jour de leur état en temps réel grâce à `handle_info/3`

3. CP1 — Installer et configurer son environnement de travail en fonction du projet

3.1 Environnement de développement

Pour le développement, j'ai utilisé VSCode car c'est l'IDE que j'ai l'habitude d'utiliser. Son catalogue d'extensions m'a permis d'utiliser des aides au développement, comme ElixirLS pour l'autocomplétion Elixir, PlantUML pour la création et la visualisation de mes schémas UML, ou TODO pour garder en tête les tâches à effectuer.

Le projet repose principalement sur Elixir et Mix, son outil natif pour lequel j'ai créé une tâche personnalisée, `mix precommit` afin de grouper les commandes de formatage, de lint, de build et d'exécution des tests afin de m'assurer de la qualité de mon code avant de le push sur la branche distante. Toutes ces étapes sont les mêmes que les tâches exécutées sur la pipeline CI à la création d'une pull request, me permettant de gagner du temps en capturant les problèmes plus tôt dans mon workflow.

`mix.setup` permet d'installer le projet et ses dépendances, créer les bases de données et jouer leurs migrations. On a deux bases de données par défaut, la base de prod (dev en local) et de tests, permettant d'effectuer des tests reproductible sur cette dernière.

Enfin, `mix phx.server` permet de lancer l'application sur le port 4000 de la machine hôte.

3.2 Outils de gestion des versions et de collaboration

Même si j'ai travaillé seul sur ce projet, j'ai appliqué les bonnes pratiques utilisées au quotidien chez Frixel afin de maintenir un historique propre, traçable et permettant le travail en équipe.

Chaque mise à jour commence par la création d'un ticket Jira s'il n'existe pas déjà, puis par la création d'une branche GIT selon la convention de nommage établie chez Frixel. Cette convention permet de relier directement le ticket, la branche et les merges sur main, eux même nommés en suivant une convention similaire.

`LQ-XX type(description)` :

- `LQ-XX` correspond au numéro de ticket JIRA
- `type` correspond à la nature de la modification (feat, fix, refactor...)
- `description` est un court résumé des changements apportés par la mise à jour

Une fois les modifications terminées, je push mon code sur une nouvelle branche distante sur GitHub puis je crée une Pull Request vers `main` ce qui déclenche automatiquement le pipeline CI grâce à GitHub Actions.

Cette pipeline exécute trois vérifications primordiales : le formatage du code avec `mix format`, le lint avec `mix credo` qui applique des règles de lisibilité et de complexité définies en amont, l'exécution des tests avec `mix test`.

Toutes ces étapes permettent de garantir de conserver un historique et une branche `main` propre, où chaque commit est validé en amont.

3.3 Conteneurisation

Dans le cadre de la conteneurisation de ce projet, j'ai été amené à créer un `Dockerfile` et un `docker-compose.yml` qui permettent de lancer l'application et sa base de données dans des conteneurs isolés, sans aucune installation locale d'Elixir ou de PostgreSQL.

Le `Dockerfile` part de l'image officielle `elixir:latest`. Grâce aux commandes fournies par `Mix` qu'on a évoqué plus tôt, il :

- Installe Hex et Rebar (`mix local.hex && mix local.rebar`), qui sont les gestionnaires de paquets Elixir
- Récupère les dépendances du projet (`mix deps.get`)
- Télécharge les assets frontend (`mix assets.setup`)
- Compile et minifie ces assets (`mix assets.deploy`) pour produire les fichiers statiques prêts pour la production
- Compile l'application complète en environnement de production (`MIX_ENV=prod mix compile`)

Lorsque le conteneur est démarré, il joue les fichiers de migration (`mix ecto.migrate`) puis lance l'application (`mix phx.server`).

Ce Dockerfile sera lui même lancé à l'aide d'une orchestration docker-compose aux côtés de la base de données pour s'assurer de la reproductibilité.

L'orchestration fonctionne comme suit :

- Le service `db` est créé sur la base de l'image officielle `postgres:16` , avec un volume persistant.
- Un healthcheck vérifie que PostgreSQL est lancé et accepte les connexions.
- Le service `app` est annoté comme dépendant de la condition de santé du service `db`
- Il est construit à partir du DockerFile local, et charge les variables d'environnement depuis un fichier `.env`.
- La variable `DATABASE_URL` est injectée directement pour pointer vers le service `db` via son nom de service Docker.

Cette configuration permet de lancer l'environnement complet avec une seule commande (`docker compose up`), ce qui garantit la reproductibilité de l'opération indépendamment de la machine hôte.

4. CP2 — Développer des interfaces utilisateur

4.1 Conception des interfaces avec Phoenix LiveView

Lifequest repose entièrement sur Phoenix LiveView pour la construction de ses interfaces. Ce modèle se distingue des approches frontend classiques (React, Vue) en maintenant l'état de l'interface côté serveur : le navigateur établit une connexion WebSocket persistante avec le serveur, qui calcule les changements de DOM et envoie uniquement les diffs nécessaires au client. Cette architecture élimine le besoin d'un framework JavaScript séparé et d'une API entre le front et le back, tout en offrant une réactivité comparable à celle d'une Single Page Application.

Chaque LiveView suit une structure définie par trois callbacks principaux :

- `mount/3` initialise l'état de la page en chargeant les données nécessaires dans les assigns.
- `handle_event/3` traite les interactions utilisateur comme les clics ou les soumissions de formulaire et met à jour l'état en conséquence.
- `handle_info/3` reçoit les messages asynchrones, notamment ceux émis via PubSub.

Le template HEx associé reflète automatiquement chaque changement d'état sans rechargement de page.

Il est possible de factoriser les composants réutilisables dans des modules personnalisés, pour permettre une organisation propre à chaque équipe.

L'implémentation de base de Phoenix les regroupe dans `core_components.ex`, qui expose des composants fonctionnels tels que `<.input>`, `<.form>` et `<.icon>`. Cette centralisation garantit la cohérence visuelle et comportementale à travers l'application : un champ de formulaire se comporte et s'affiche de manière identique partout, et toute modification s'applique en un seul point.

Exemple concret de la LiveView `dashboard_live` : Voir annexe 2.1

A l'initialisation de la LiveView, la fonction `mount/3` est appelée, et appelle elle-même les différentes fonctions du contexte permettant de récupérer les données en base permettant l'affichage des données sur le dashboard, avant de les placer dans les assigns. Parmi elles, la fonction `load_dashboard_data` qui vient appeler différentes fonctions du contexte `Finances` pour récupérer les revenus et les dépenses de l'utilisateur en base. Cela permet de générer les deux graphiques. Si des revenus ou des dépenses récurrentes ont été générés automatiquement en base sans avoir été validés, elles apparaissent dans une section de la page, `pending_recurring_section`, permettant à l'utilisateur de les valider une par une. En cliquant sur le bouton valider, l'évènement `validate_recurring` est déclenché et capté par un `handle_event` qui va lui-même essayer d'appeler la fonction `validate_recurring/3` du contexte `Finances` pour mettre à jour les informations dans les assigns de la socket. La

transaction perd son état pending, elle disparaît donc de la section dédiée à ces dernières.

4.2 Charte graphique et responsive design (Tailwind CSS / daisyUI)

Pour la couche de présentation, j'ai utilisé Tailwind CSS v4 associé à daisyUI, qui sont les outils fournis par Phoenix et que j'utilise au quotidien chez Frixel.

Tailwind est un framework CSS utility-first : plutôt que de définir des classes sémantiques, on compose les styles directement dans le HTML grâce à des classes atomiques (`flex` , `gap-4` , `rounded-lg` ...). Cela évite d'écrire du CSS custom et élimine les conflits de nommage.

A la compilation, seules les classes effectivement utilisées sont incluses dans le bundle final, ce qui allège le poids des assets en production.

daisyUI vient compléter Tailwind en proposant des composants prêts à l'emploi : `btn` , `card` , `badge` , `menu` ... Ce sont de simples classes CSS qui regroupent des combinaisons de classes Tailwind, sans JavaScript embarqué.

Le système de thèmes est configuré dans `app.css` où l'on définit nos couleurs `primary` , `base-100` , `base-200` ... Changer le thème revient à changer la valeur de ces variables, sans toucher aux composants. Lifequest propose trois modes : clair, sombre, et système. Ce dernier détecte automatiquement la préférence du navigateur.

La bascule est gérée côté client via l'attribut `data-theme` sur la balise `html` , ce qui évite un rechargement de page.

Pour le responsive, j'ai utilisé les préfixes de breakpoints de Tailwind (`sm:` , `md:` , `lg:`), qui permettent de modifier les valeurs voulues selon la taille de l'écran inline, sans avoir à écrire de media-queries.

4.3 Accessibilité (RGAA)

Pour travailler sur l'accessibilité, je me suis basé sur les retours d'audit de Lighthouse sur chacune des pages de l'application, ce qui m'a amené à étudier les attributs aria manquants et les contrastes de couleur.

Les composants daisyUI fournissent des labels aria de base sur les éléments courants. J'ai ajouté les labels manquants sur les boutons sans texte.

De plus, sur certains éléments qui se répètent sur une même page, j'ai ajouté un label dynamique. Par exemple sur les icônes de modification et de suppression des différentes transactions de la page "Finances" j'ai inclu le libellé de la transaction ciblée, ce qui donne des labels comme "Modifier Loyer" ou "Supprimer Salaire net", permettant à l'utilisateur de distinguer chaque bouton sans ambiguïté.

Toutes les pages de l'application ont un score d'accessibilité supérieur à 90 suite aux audits Lighthouse.

Certaines interactions complexes comme les graphiques n'ont pas fait l'objet de tests spécifiques avec des lecteurs d'écran.

4.4 Conformité RGPD — Mentions légales

Lifequest collecte et traite des données personnelles, ce qui implique de respecter le Règlement Général sur la Protection des Données. J'ai créé une page dédiée, accessible depuis le footer sur l'ensemble de l'application, y compris sans être connecté.

La base légale retenue est l'exécution d'un contrat au sens de l'article 6(1)(b) du RGPD : en créant un compte, l'utilisateur accepte que ses données soient traitées pour lui permettre d'utiliser le service. Les données sont conservées pendant toute la durée d'activité du compte, puis supprimées à sa résiliation. Les tokens d'authentification expirent automatiquement après 60 jours.

La page liste les droits de l'utilisateur : accès, rectification, effacement, portabilité et opposition. Le droit à l'effacement est directement accessible depuis les paramètres du compte. La suppression du compte entraîne la suppression en cascade de toutes les données associées en base.

4.5 Tests des composants d'interface

Les interfaces de Lifequest sont testées via le module `Phoenix.LiveViewTest` qui simule le cycle de vie complet d'une LiveView dans un environnement de test.

Les tests utilisent le module `ConnCase` pour établir une connexion HTTP simulée avec une session authentifiée.

La stratégie de test porte sur trois aspects : la vérification du rendu HTML attendu, la présence des éléments d'interface critiques et le comportement en réponse aux interactions utilisateur.

`describe` permet de grouper des tests unitaires qui partagent la même situation initiale `test` est un test unitaire, qui contient nos assertions `assert html =~ "texte attendu"` permet de vérifier la présence d'un contenu textuel dans le rendu `assert has_element?(live_view, "sélecteur CSS")` permet de vérifier la présence d'un élément spécifique dans le DOM.

Exemple concret des tests de la LiveView `dashboard_live` : Voir annexe 2.1

`setup :register_and_log_in_user` permet de simuler la situation initiale : un utilisateur est connecté. C'est nécessaire car toutes les opérations testées ne sont déclenchables que dans cette situation précise.

`create_recurring_last_month(scope, attrs \\ %{})` est une fonction qui permettra de créer une opération récurrente au besoin. Des valeurs par défaut sont définies, mais peuvent être écrasées en fournissant le deuxième argument facultatif.

`test "shows pending recurring incomes from last month"` crée une opération récurrente à valider par défaut et s'assure qu'elle s'affiche bien sur la page

`test "does not show recurring already validated this month"` crée une opération récurrente et insère sa validation en base pour le mois courant, puis vérifie qu'elle n'apparaît bien pas dans la liste des revenus à valider

`test "validate_recurring duplicates transaction to current month"` simule la validation manuelle par l'utilisateur, et s'assure que cela entraîne la disparition de la ligne sur la page

4.6 Sécurité des interfaces (XSS, CSRF)

Phoenix fournit des protections contre les principales vulnérabilités web directement dans sa couche de templating. Contre le Cross-Site Scripting (XSS), HEx échappe automatiquement toute variable interpolée via `{@assign}` : si une donnée utilisateur contient du HTML ou du JavaScript, le contenu est affiché tel quel, sans être interprété par le navigateur.

Aucune balise `<script>` inline n'est présente dans les templates HEx de Lifequest : l'ensemble du JavaScript est isolé dans le répertoire `assets/js/`.

Contre les attaques Cross-Site Request Forgery (CSRF), Phoenix génère un token unique intégré à chaque formulaire via le composant `<.form>`. Ce token est vérifié côté serveur à chaque soumission, ce qui garantit que la requête provient bien de l'application et non d'un site tiers.

Les Scopes et leur présence dans les arguments des fonctions de Repo garantissent que l'utilisateur actif peut récupérer les données lui appartenant, et seulement celles-ci.